# Angular Lifecycle & Component Communication Exam Prep

Practice questions for Angular lifecycle hooks, component communication, signals, bindings, and FormsModule.

## Part 1 — Multiple Choice Questions

**1. What does lifecycle mean in Angular?**

A) CSS animation

B) Stages of component existence

C) Angular routing

D) TypeScript compilation

**2. Which hook is mainly used for initialization logic?**

A) constructor

B) ngOnDestroy

C) ngOnInit

D) ngDoCheck

**3. Which hook is called before component destruction?**

A) ngOnInit

B) ngOnDestroy

C) ngAfterViewInit

D) ngOnChanges

**4. Which hook reacts to input changes?**

A) ngOnChanges

B) ngOnDestroy

C) constructor

D) ngAfterViewChecked

**5. What object does ngOnChanges receive?**

A) EventEmitter

B) Observable

C) SimpleChanges

D) Signal

**6. What is constructor mainly used for in Angular?**

A) Cleanup

B) Dependency Injection

C) Rendering HTML

D) Routing

**7. Which hook is better for HTTP requests and data loading?**

A) constructor

B) ngOnInit

C) ngOnDestroy

D) ngAfterViewInit

**8. Which hook is commonly used for cleanup?**

A) ngOnChanges

B) constructor

C) ngOnDestroy

D) ngOnInit

**9. What function is commonly used inside ngOnDestroy for timers?**

A) clearTimeout

B) removeSignal

C) clearInterval

D) stopTimer

**10. What can happen if cleanup is NOT done in ngOnDestroy?**

A) Faster rendering

B) Memory leaks

C) CSS errors

D) TypeScript compilation errors

## Part 2 — Fill in the Blanks

**1.**

```
export class StudentComponent implements __________ {}
```

**2.**

```
export class TimerComponent implements OnInit, __________ {}
```

**3.**

```
ngOnChanges(changes: __________): void {}
```

**4.**

```
__________(this.intervalId);
```

**5.**

```
changes['grade'].________________
```

**6.**

```
changes['grade'].________________
```

**7.**

```
changes['grade'].________________
```

**8.**

```
________________(): void {}
```

9.
```
_________________(private api: StudentService) {}
```

10.
```
student = ___________<Student>();
```

11.
```
deleteStudent = ___________<number>();
```

12.
```
this.deleteStudent.___________(id);
```

13.
```
(delete)="removeStudent(___________)"
```

14.
```
<input ___________="searchTerm">
```

15.
```
imports: [CommonModule, ___________]
```

16.
```
@___________ (isLoggedIn)
```

17.
```
@___________ (student of students)
```

18.
```
count = ___________(0);
```

19.
```
double = ___________(() => this.count() * 2);
```

20.
```
this.count.___________(n => n + 1);
```

## Part 3 — Open Questions

**1. Explain the difference between constructor() and ngOnInit().**

**2. Why is ngOnInit preferred for HTTP requests and data loading?**

**3. What does ngOnChanges() do?**

**4. Explain currentValue, previousValue, and firstChange.**

**5. What is ngOnDestroy() used for?**

**6. What can happen if cleanup is not done in ngOnDestroy()?**

**7. Explain setInterval() and clearInterval().**

**8. When is ngAfterViewInit() called?**

**9. Describe Angular lifecycle order.**

**10. Explain: Data flows DOWN, events flow UP.**

# Answers

## Multiple Choice Answers

1. B
2. C
3. B
4. A
5. C
6. B
7. B
8. C
9. C
10. B

## Fill in the Blanks Answers

1. OnInit
2. OnDestroy
3. SimpleChanges
4. clearInterval
5. currentValue
6. previousValue
7. firstChange
8. ngAfterViewInit
9. constructor
10. input
11. output
12. emit
13. $event
14. [(ngModel)]
15. FormsModule
16. if
17. for
18. signal
19. computed
20. update

## Open Question Key Points

- constructor is mainly for dependency injection; ngOnInit is for initialization logic and data loading.
- ngOnInit runs when Angular is fully ready and inputs are available.
- ngOnChanges reacts to @Input changes.
- currentValue = new value, previousValue = old value, firstChange = first update check.
- ngOnDestroy is used for cleanup before component removal.
- Without cleanup, memory leaks can occur.
- setInterval creates repeating timer; clearInterval stops it.
- ngAfterViewInit runs after template/view initialization.
- Typical order: constructor → ngOnChanges → ngOnInit → ngAfterViewInit → ngOnDestroy.
- Data flows from parent to child via input(); events flow up via output() and emit().